Building Dynamic, Intelligent Web Based Solutions with ASP.NET

I T W P 2 3 0 0

Paul C. Sbraccia, MA

Macomb Community College

2012

V3.4, 2017

Table of Contents

First Day Activities **1**

Homework 1 ..... **11**

Homework 2 ..... **27**

Homework 3 ..... **37**

Homework 4 ..... **43**

Homework 5 ..... **55**

An Online eBook Book Store

Project 1 ..... **71**

Project 2 ..... **85**

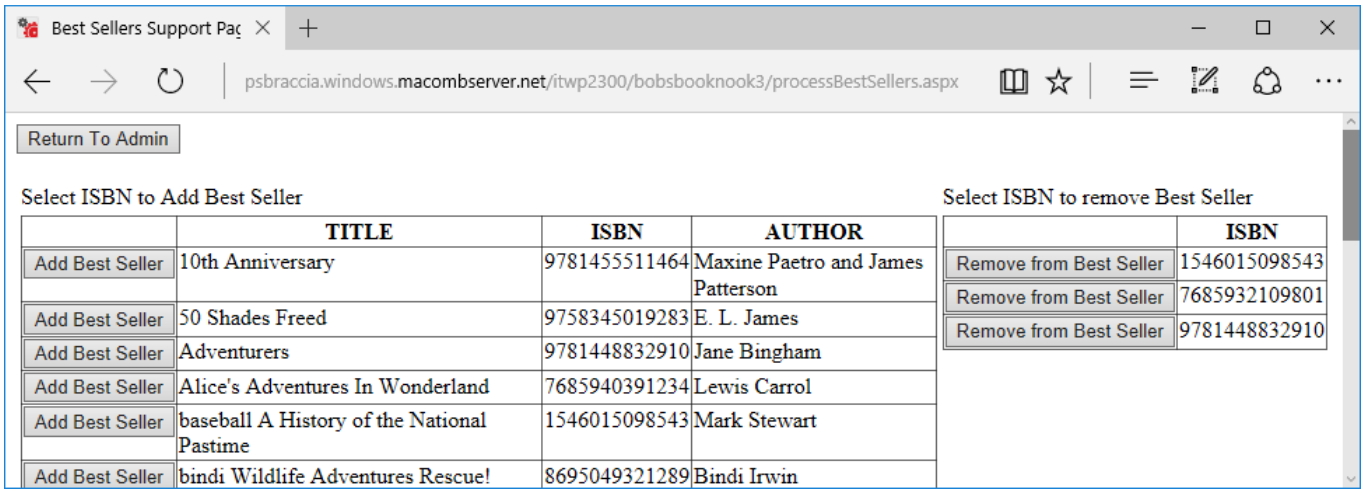
Project 3 ..... **95**

Project 4 ..... **107**

Project 5

ITWP 2300

BUILDING DYNAMIC, INTELLIGENT WEB BASED SOLUTIONS WITH ASP.NET



PROJECT 3

In this unit we will enable the ability to modify the best-sellers list (table) from the admin page and remove books from the inventory. We will modify Default.aspx to display best sellers and continue work on our shopping cart.

ITWP 2300 eBook Store Project 3

In the Monday Morning Standup:

- The Project Manager complemented your group on the design and functionality of both the Admin page and the Default page.
- S/He also mentioned the brilliant idea of saving ownerID to a file to prevent login deficiencies which would mess up the database table name issue.
- S/He also asked if you needed ownerID on many of the pages, should it not go in a class file and be referenced once rather than being copied 5 or 6 times by the time the project is finished?
- This week we need you to concentrate on two issues and to more fully develop the shopping cart.
- 1) In Admin add the ability to add or delete an ISBN from table ownerID+"BESTSELLER". "Might I suggest that you place a button under View Inventory named Process BestSeller which links to a page where you will be able to add and delete best sellers from the list".
    - Be sure to add a [Back to Admin] button.
    - The addition of Bestsellers leads to work in Default.aspx: we need a way to view Best Sellers.
- 2) Add the ability to remove an item from table ownerID_BOOKS. You can probably manage an additional button being added to the Admin gvDisplay with the text of "DELETE".
- On the ShoppingCart page
    - display each book that has been selected.
    - The gridview will show the image, title, author and ISBN.
    - You will add a button to "Place back on the shelf" which removes the record from the shopping cart gridview by removing it from the sessions variable.
- Add a Checkout button which for now, clears the items in sessions variables and the gridview and returns the browser back to Default.aspx.

First Things First

Make a copy of BBN2 and name it BobsBookNook3.
Set Admin.aspx as the Start Page.
Make sure the code works as is expected.

On to the Details!

In Admin add the ability to add or delete an ISBN from table ownerID+"BESTSELLER". "Might I suggest that you place a second button under View Inventory named Process BestSeller which links to a page where you will be able to add and delete best sellers from the list".

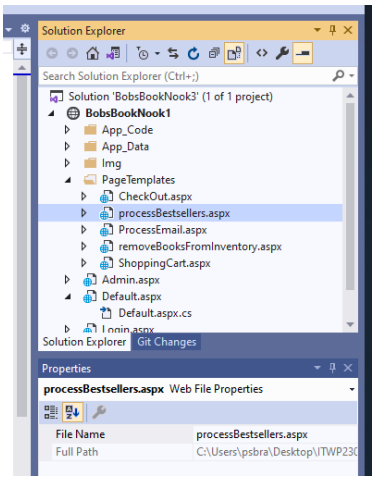
Before we write one new line of code or add any new buttons, we have to check that the table ownerID + "BESTSELLER" exists. The code shown below is added to the bottom of Admin.Page_Load, just after checkTables for BOOKS, just before the call to **initLstProgress()**;

```
checkTables(txtOwnerID.Text + "BOOKS", "CREATE TABLE " + txtOwnerID.Text + "BOOKS (Title CHAR(64),
ISBN CHAR(14) UNIQUE, Author CHAR(64), Category CHAR(48), Image CHAR(128), Description CHAR(255))");
```

```
checkTables(txtOwnerID.Text + "BESTSELLER", "CREATE TABLE " + txtOwnerID.Text +
"BESTSELLER (ISBN CHAR(20))");
```

```
initLstProgress();
```

Ed Note: Be sure to distinguish in your mind between BESTSELLER the table name and the code references to bestsellers (plural).



Ed Note: Everything seems ok now and we can create the bestsellers table the first time we execute Admin.aspx. However, when we run Admin.aspx a second time, there is heartache because the gvCatalog contains a call to the image processing function.

The easiest way to 'fix' this is to create a second, but invisible grid view object named gvHidden with no special calls or formatting as we have for gvCatalog. We will then refer to gvHidden when executing the checkTable function.

Add gvHidden. You can place it anywhere on the page you want as it will never be seen. Set the visibility property to False.

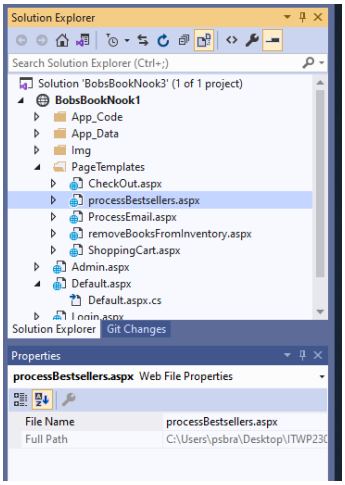
Modify checkTables as shown below.

```
private void checkTables(string Table, string sqlStatement)
{
        //Response.Write(Table + "    " + sqlStatement + "<br/>");
        //throw new NotImplementedException();
        lblErrorMessage.Text = "";

        myDatabaseConnection.executeSQL("SELECT * FROM " + Table, ref gvHidden, ref lblErrorMessage);

        //turn off gridview
        gvHidden.Visible = false;
        //check for error message: create the connection if necessary
        if (lblErrorMessage.Text.Length > 0)
        {
                lblErrorMessage.Text = "";
                myDatabaseConnection.executeSQL(sqlStatement, ref gvHidden, ref lblErrorMessage);
        }

}
```

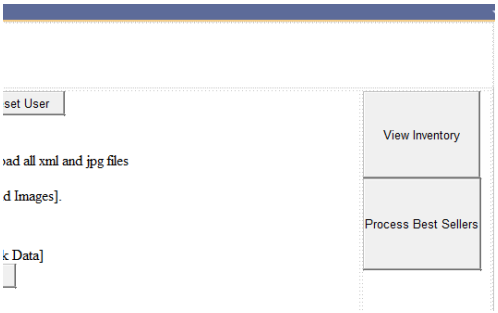
Adding processBestSellers.aspx

Utilizing the template provides us access to site specifics setup criterion rather than looking in a "code book" to see how to set up a project. In the last unit, we moved Default.aspx from the PageTemplates folder over to the working directory which contains Admin.aspx and LogIn.aspx.

Rather than having to go through the hassle of creating a new page, simply drag and drop processBestsellers.aspx from the template directory to the working directory.
(In this case, the bolded BobsBookNook1 Project.)



Let's place a button on **Admin.aspx** just under View Inventory. Set the name to "Process Best Sellers", and the ID to btnProcessBestSellers.



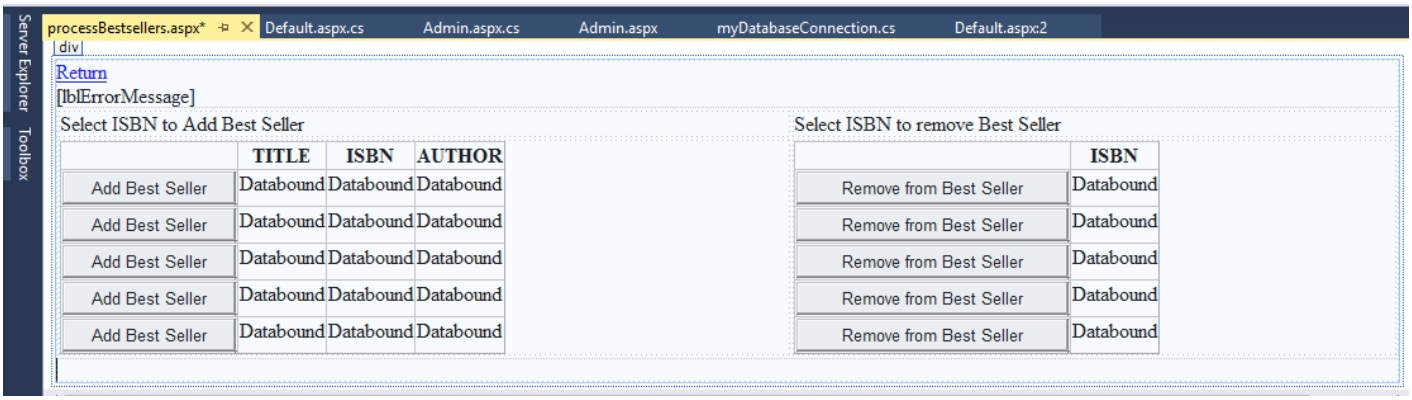
Place the following bolded code in the code for btnProcessBestSellers.

```
protected void btnProcessBestSellers_Click(…
    {
        Response.Redirect("processBestsellers.aspx");
    }
```

Executing the code at this time will bring up a blank page.

Coding processBestsellers.aspx

In the set of documents that you were given at the meeting, the following "sketch" was included titled "ProcessBestSellers".



From Top to Bottom, Left to Right:

<title>Process Best Sellers</title>

Button: Text: **Return to Admin** Action: **Response.Redirect("Admin.aspx");**

Label: **lblErrorMessage** Text: ""

Table (2x2)

Select ISBN to Add Best Seller	Select ISBN to Remove Best Seller
gvCatalog : DataKeyNames **ISBN**	**gvBestSellers** : DataKeyNames **ISBN**

After a team conference: You all agreed that ISBN was not appropriate in the sketch especially if we use buttons. Let's design it with [Button] rather than ISBN and see how it plays.

Let's take a look at how to set up gvCatalog. We want to show every item in the inventory, ordered by Title. We only need to display Title, ISBN and Author.

Working in processBestsellers.aspx.cs: add the following set of commands to Page_Load.

```
string sqlCommand = "SELECT ISBN, TITLE, AUTHOR FROM " + ownerID + "BOOKS ORDER BY TITLE";
myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
```

You alertly notice that ownerID is flagged as missing.

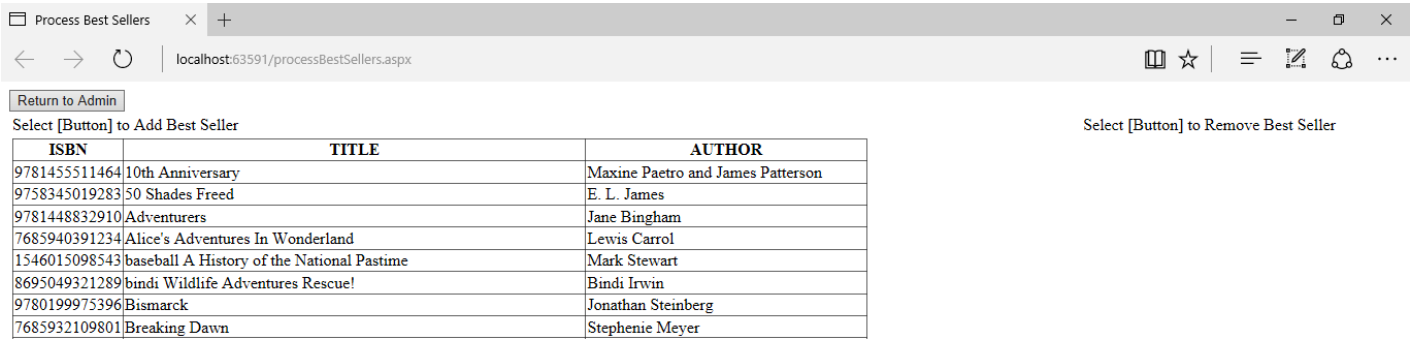
Set a global string variable `string ownerID;`

If you remember from Project 2, we saved a copy of the ownerID in a text file. Add the bolded code from Default.aspx to Page Load, making sure the sql statements are after the catch block.

```
protected void Page_Load(…
{
     string uploadPath = Server.MapPath("~/App_Data/");
    try
    {
    ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
    ownerID = ownerID.Trim();
    //Response.Write(ownerID + "<br />"); //error checking only.
   }
   catch { }
   //Load gvCatelog
   string sqlCommand = "SELECT ISBN, TITLE, AUTHOR FROM " + ownerID + "BOOKS ORDER BY TITLE";
   myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
}
```

We can add the code for gvBestSellers next:

```
//gvCatalog
string sqlCommand = "SELECT ISBN, TITLE, AUTHOR FROM " + ownerID + "BOOKS ORDER BY TITLE";
myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);

//gvBestsellers
sqlCommand = "SELECT ISBN FROM " + ownerID + "BESTSELLER ORDER BY ISBN";
myDatabaseConnection.executeSQL(sqlCommand, ref gvBestSellers, ref lblErrorMessage);
```



ISBN	TITLE	AUTHOR
9781455511464	10th Anniversary	Maxine Paetro and James Patterson
9758345019283	50 Shades Freed	E. L. James
9781448832910	Adventurers	Jane Bingham
7685940391234	Alice's Adventures In Wonderland	Lewis Carrol
1546015098543	baseball A History of the National Pastime	Mark Stewart
8695049321289	bindi Wildlife Adventures Rescue!	Bindi Irwin
9780199975396	Bismarck	Jonathan Steinberg
7685932109801	Breaking Dawn	Stephenie Meyer

Since we are working with a table, now might be the best time to add

```
td {
        vertical-align: top;
   }
```

To the style code block.

We can now edit the columns for gvCatalog and gvBestSeller creating a command button and three boundfields for gvCatalog and one command button and one boundfield for gvBestseller.

We should be familiar enough to manage this by ourselves. If you are, continue on past the figure below. If not, let's see what we need to do:

gvCatalog

Open **gvCatalog** to **Edit Columns**.

Add CommandField Button
3 Bound Fields.

For the CommandField: **Button Type** = **Button**
Select Text = Add to Best Sellers
ShowCancelButton = **False**
ShowSelectButton = **True**
First Bound Field: **Header Text** = **Title**
DataField = Title
Second Bound Field: **Header Text** = **Author**
DataField = Author
Third Bound Field: **Header Text** = **ISBN**
DataField = ISBN

Un-check Auto-generate fields.

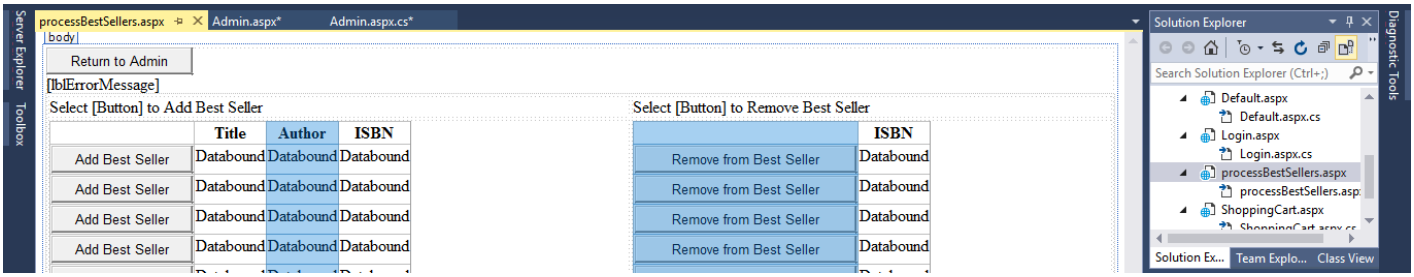
Open **gvCatalog** to **Edit Columns**.

Add CommandField Button
1 Bound Field.

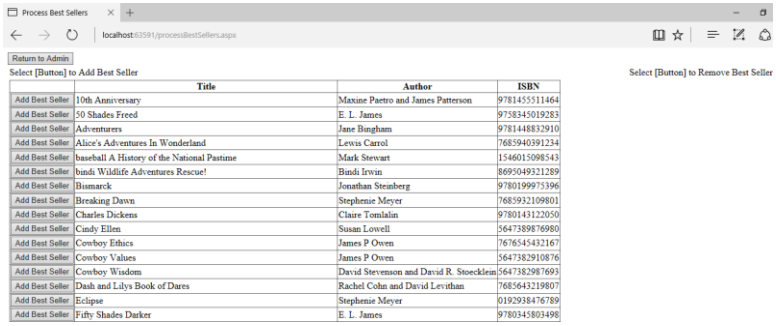
For the CommandField: **Button Type** = **Button**
Select Text = Remove
ShowCancelButton = **False**
ShowSelectButton = **True**

Bound Field: **Header Text** = **ISBN**
DataField = ISBN

In both cases remember to turn off **[ ] Auto-generate Fields**



Ed Note: Need help? Remember that we did this back in Project 2 Page 7, modifying the menu gridviews in Default.aspx.



	Title	Author	ISBN
Add Best Seller	10th Anniversary	Maxine Paetro and James Patterson	9781455511464
Add Best Seller	50 Shades Freed	E. L. James	9758345019283
Add Best Seller	Adventurers	Jane Bingham	9781448832910
Add Best Seller	Alice's Adventures In Wonderland	Lewis Carrol	7685940391234
Add Best Seller	baseball A History of the National Pastime	Mark Stewart	1546015098543
Add Best Seller	bindi Wildlife Adventures Rescue!	Bindi Irwin	8695049321289
Add Best Seller	Bismarck	Jonathan Steinberg	9780199975396
Add Best Seller	Breaking Dawn	Stephenie Meyer	7685932109801
Add Best Seller	Charles Dickens	Claire Tomlalin	9780143122050
Add Best Seller	Cindy Ellen	Susan Lowell	5647389876980
Add Best Seller	Cowboy Ethics	James P Owen	7676545432167
Add Best Seller	Cowboy Values	James P Owen	5647382910876
Add Best Seller	Cowboy Wisdom	David Stevenson and David R. Stoecklein	5647382987693
Add Best Seller	Dash and Lilys Book of Dares	Rachel Cohn and David Levithan	7685643219807
Add Best Seller	Eclipse	Stephenie Meyer	0192938476789
Add Best Seller	Fifty Shades Darker	E. L. James	9780345803498

Running.

Do not be concerned that best sellers are not showing, we do not have any as of yet.

Adding and Deleting Best Sellers:

Double check that ISBN is set as the Key Data Item for both gvCatalog and gvBestSeller. Double click on gvCatalog and gvBestSellers to create the SelectedIndexChanged methods associated with each gridview.

gvCatalog:

```
protected void gvCatalog_SelectedIndexChanged(…
    {
        //Response.Write(gvCatalog.SelectedValue.ToString());
        string sqlCommand = "INSERT INTO " + ownerID + "BESTSELLER VALUES('" +
gvCatalog.SelectedValue.ToString() + "')";

       // Response.Write(sqlCommand + "<br/>");
        myDatabaseConnection.executeSQL(sqlCommand, ref gvBestSellers, ref lblErrorMessage);

        sqlCommand = "SELECT ISBN FROM " + ownerID + "BESTSELLER ORDER BY ISBN";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvBestSellers, ref lblErrorMessage);
    }
```

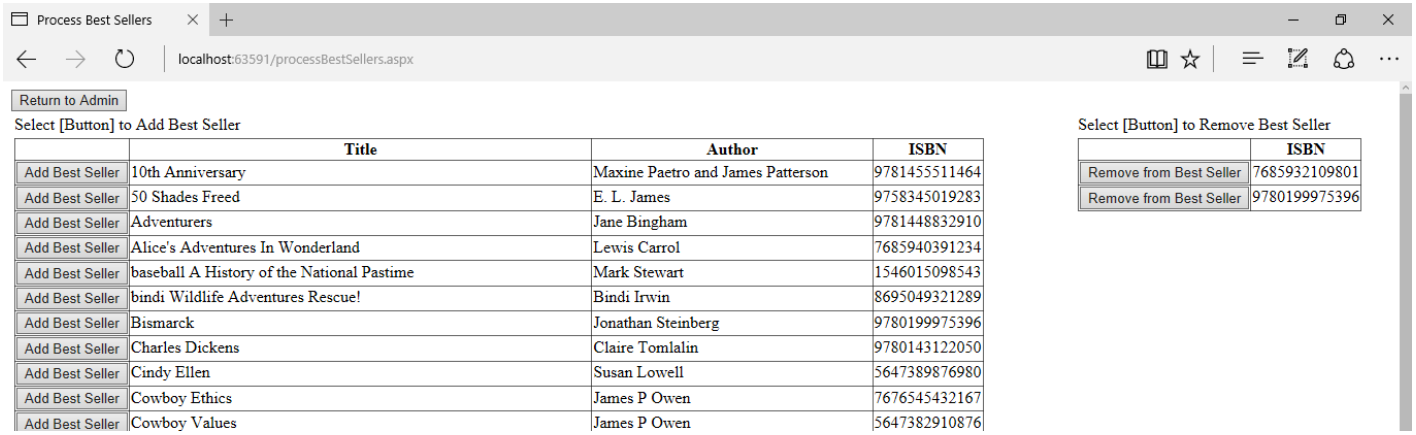
gvBestSellers:

```
protected void gvBestSellers_SelectedIndexChanged(…
{
    string sqlCommand = "DELETE FROM " + ownerID + "BESTSELLER WHERE ISBN = ('" +
gvBestSellers.SelectedValue.ToString() + "')";
    myDatabaseConnection.executeSQL(sqlCommand, ref gvBestSellers, ref lblErrorMessage);

    sqlCommand = "SELECT ISBN FROM " + ownerID + "BESTSELLER ORDER BY ISBN";
    myDatabaseConnection.executeSQL(sqlCommand, ref gvBestSellers, ref lblErrorMessage);
}
```

Note: As always, each command is on one line of code.

IF you cannot see the BestSellers gridview, make sure you included the td code in the CSS style code block in processBestSellers.aspx.

We will always refresh gvBestSellers but we do not have to refresh gvCatalog, as it will not change.



	Title	Author	ISBN
Add Best Seller	10th Anniversary	Maxine Paetro and James Patterson	9781455511464
Add Best Seller	50 Shades Freed	E. L. James	9758345019283
Add Best Seller	Adventurers	Jane Bingham	9781448832910
Add Best Seller	Alice's Adventures In Wonderland	Lewis Carrol	7685940391234
Add Best Seller	baseball A History of the National Pastime	Mark Stewart	1546015098543
Add Best Seller	bindi Wildlife Adventures Rescue!	Bindi Irwin	8695049321289
Add Best Seller	Bismarck	Jonathan Steinberg	9780199975396
Add Best Seller	Charles Dickens	Claire Tomlalin	9780143122050
Add Best Seller	Cindy Ellen	Susan Lowell	5647389876980
Add Best Seller	Cowboy Ethics	James P Owen	7676545432167
Add Best Seller	Cowboy Values	James P Owen	5647382910876

	ISBN
Remove from Best Seller	7685932109801
Remove from Best Seller	9780199975396

Having added three books and deleting one from Best Sellers.

processBestSellers is now complete. Be sure to comment out all grey using statements. Next up, seeing Best Sellers in Default.aspx!

Seeing Best Sellers in Default.aspx

A bit of application process discussion by the team over coffee:

"You know, when looking at the actual book store, we see the authors, but we have to scroll to even know that there is a category menu."

"Let's just move the category menu up to the right of Author."

"Now, what about these best sellers? All we have is a list of ISBNs."

"I was thinking about this project in my sleep."

"Don't we all!"

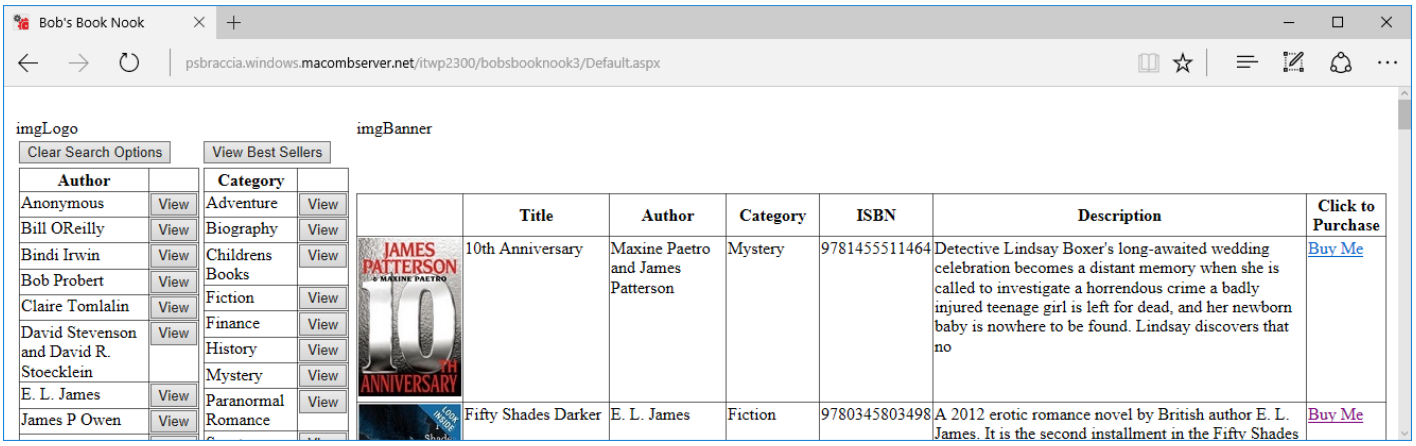
"But I came up with the idea of two additional buttons. One to reset the catalog display after they selected an author or category or they would have to guess who or what they needed to look for next."

"Sounds good and the other?"

"A button that will display all the best sellers in gvCatalog."

"You know, that just might work. Let's surprise the boss at the next standup."

Working in Default.aspx:



Author	
Anonymous	View
Bill OReilly	View
Bindi Irwin	View
Bob Probert	View
Claire Tomlalin	View
David Stevenson and David R. Stoecklein	View
E. L. James	View
James P Owen	View

Category	
Adventure	View
Biography	View
Childrens Books	View
Fiction	View
Finance	View
History	View
Mystery	View
Paranormal Romance	View

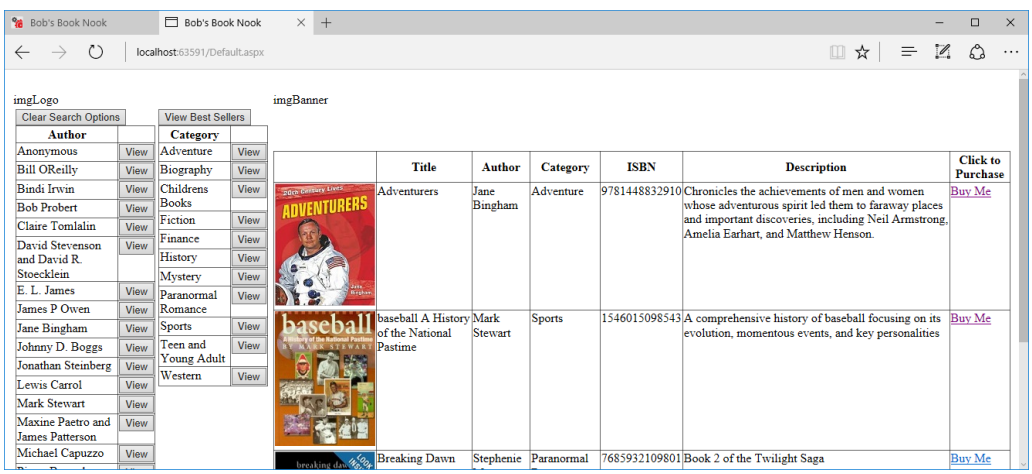
	Title	Author	Category	ISBN	Description	Click to Purchase
	10th Anniversary	Maxine Paetro and James Patterson	Mystery	9781455511464	Detective Lindsay Boxer's long-awaited wedding celebration becomes a distant memory when she is called to investigate a horrendous crime a badly injured teenage girl is left for dead, and her newborn baby is nowhere to be found. Lindsay discovers that no	Buy Me
	Fifty Shades Darker	E. L. James	Fiction	9780345803498	A 2012 erotic romance novel by British author E. L. James. It is the second installment in the Fifty Shades	Buy Me

- Insert a new column to the right of the table containing gvAuthor and gvCategory.
- Drag gvAuthor to Row 2 Column 1.
- Drag gvCategory to Row 2, Column 2.
- Insert buttons in Row 1 Column 1 and Row 1 Column 2.
- Set one text value to **Clear Search Options**, the other to **View Best Sellers**. Make sure to name them appropriately.
- The code behind Clear Search Options is nothing more than the original sqlCommand to load gvCatalog.

```
protected void btnClearSearchOptions_Click(…
{
    string sqlCommand = "SELECT Title, ISBN, Author, Category, Image, Description FROM " + ownerID +
"BOOKS";

    myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
}
```

The code behind the View Best Seller button uses the list of ISBNs and an inner join to create the sql command to load gvCatalog.

```
string sqlCommand = "SELECT " + ownerID + "BOOKS.Title, " + ownerID + "BOOKS.ISBN, " + ownerID +
"BOOKS.Author, " + ownerID + "BOOKS.Category, " + ownerID + "BOOKS.Image, " + ownerID +
"BOOKS.Description FROM " + ownerID + "BOOKS INNER JOIN " + ownerID + "BESTSELLER ON " + ownerID
+ "BOOKS.ISBN = " + ownerID + "BESTSELLER.ISBN";

myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
```



Having clicked View Best Sellers.

Now when I click [Clear Search Options], the entire catalog is visible again.

S/He also asked if you needed ownerID on many of the pages, should it not go in a class file and be referenced once rather than being copied 5 or 6 times by the time the project is finished?

Looking at Default.aspx.cs and processBestSellers.aspx.cs, and thinking ahead to mods to shoppingCart.aspx.cs, we use the following code:

```
string uploadPath = Server.MapPath("~/App_Data/");
try
{
   ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
   ownerID = ownerID.Replace("\r\n", "");
```

```
}
catch { }
```

Knowing as I do that we will end up with a number of support pages that need this, we can create a class file and place it in App_Code right next to our myDatabaseConnection.cs code.

Click on App_Code and insert a new class file named getOwnerId.cs . Open the new cs file and add the bold code below.

```
using System;

/// <summary>
/// Summary description for getOwnerID
/// </summary>
public class getOwnerID
{
    public getOwnerID()
    {
        //
        // TODO: Add constructor logic here
        //
    }

    public static string getID(string uploadPath)
    {
        string ownerID;

        try
        {
            ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
            ownerID = ownerID.Replace("\r\n", "");
            return (ownerID);

        }
        catch { return (""); }
    }
}
```

As you can see from the above code, we use a local variable **ownerID** and a **try-catch** block to enable us to return an empty string if we cannot find the file. Modifying the file allows us to return just the **ownerID** without any extra line feeds. To call this, we will need a page level **string ownerID** and send the path to where the file is located as a parameter.

At the top of Page_Load for Default.aspx.cs and processBestSellers.aspx.cs, comment out the code as shown below and replace with the call to getID as shown below.

```
//string uploadPath = Server.MapPath("~/App_Data/");
//try
//{
//    ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
//    ownerID = ownerID.Replace("\r\n", "");
//   // Response.Write(ownerID + "<hr>");

//}
//catch { }

ownerID = getOwnerID.getID(Server.MapPath("~/App_Data/"));
```

ITWP 2300 eBook Store Project 3

Add the ability to remove an item from table owner_BOOKS. You can probably manage an additional button being added to the Admin gvDisplay with the text of "DELETE ME". This has proven not to be the case.

This is not as hard to do as one would think especially since we just did the delete from bestsellers where ISBN… yes, we will use basically the same code with different table names.

- In Admin.aspx place a button with text set to **Remove Book Titles from Category.** Place it where you think is best.
- Add the appropriate Response.Redirect code to point to the new page we are about to move from the templates folder.
- Move **removeBooksFromInventory.aspx** from the template folder to the full project**.**
- Set the **<title></title>** property to **Remove Books from Inventory.**
- Add a button named **Return to Admin**. Set the Response.Redirect code in the On_Click event to point to Admin.aspx.
- Add a label: lblErrorMessage
- Add a GridView object named **gvDeleteBooks**.
- Add a page level (global) **string ownerID;** (shown below).
- In Page_Load add the bolded code to load the GridView.

```
string ownerID;

protected void Page_Load(object sender, EventArgs e)
{
    ownerID = getOwnerID.getID(Server.MapPath("~/App_Data/"));

    string sqlCommand = "SELECT IMAGE, TITLE, ISBN FROM " + ownerID + "BOOKS";
    myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref lblErrorMessage);
}

protected void btnReturnToAdmin_Click(object sender, EventArgs e)
{
    Response.Redirect("Admin.aspx");
}
```

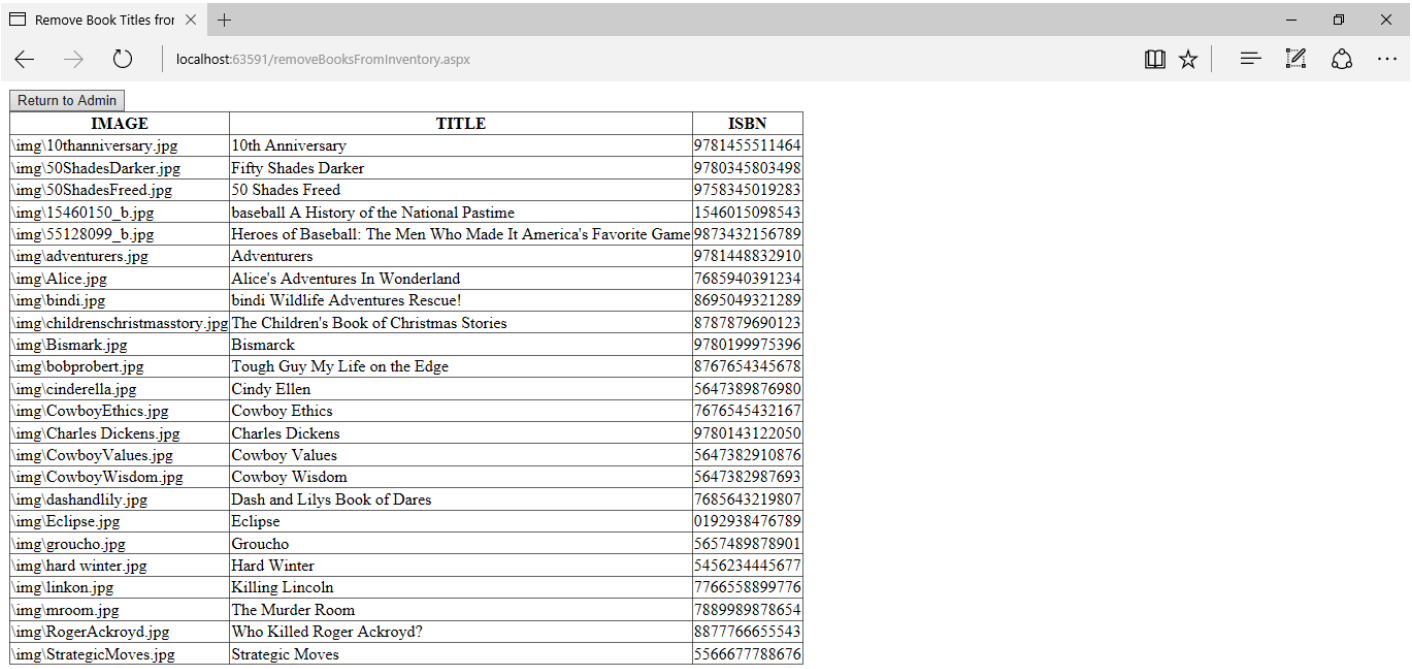
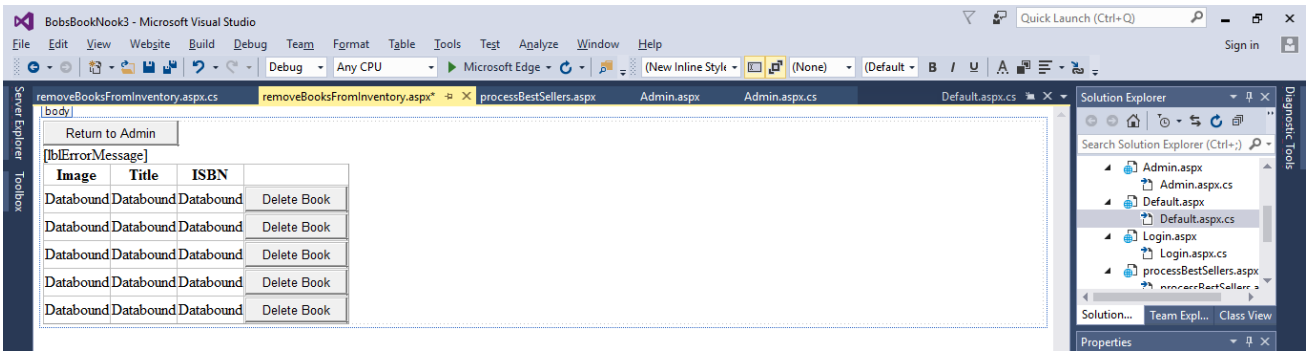


IMAGE	TITLE	ISBN
\img\10thanniversary.jpg	10th Anniversary	9781455511464
\img\50ShadesDarker.jpg	Fifty Shades Darker	9780345803498
\img\50ShadesFreed.jpg	50 Shades Freed	9758345019283
\img\15460150_b.jpg	baseball A History of the National Pastime	1546015098543
\img\55128099_b.jpg	Heroes of Baseball: The Men Who Made It America's Favorite Game	9873432156789
\img\adventurers.jpg	Adventurers	9781448832910
\img\Alice.jpg	Alice's Adventures In Wonderland	7685940391234
\img\bindi.jpg	bindi Wildlife Adventures Rescue!	8695049321289
\img\childrenschristmasstory.jpg	The Children's Book of Christmas Stories	8787879690123
\img\Bismark.jpg	Bismarck	9780199975396
\img\bobprobert.jpg	Tough Guy My Life on the Edge	8767654345678
\img\cinderella.jpg	Cindy Ellen	5647389876980
\img\CowboyEthics.jpg	Cowboy Ethics	7676545432167
\img\Charles Dickens.jpg	Charles Dickens	9780143122050
\img\CowboyValues.jpg	Cowboy Values	5647382910876
\img\CowboyWisdom.jpg	Cowboy Wisdom	5647382987693
\img\dashandlily.jpg	Dash and Lilys Book of Dares	7685643219807
\img\Eclipse.jpg	Eclipse	0192938476789
\img\groucho.jpg	Groucho	5657489878901
\img\hard winter.jpg	Hard Winter	5456234445677
\img\linkon.jpg	Killing Lincoln	7766558899776
\img\mroom.jpg	The Murder Room	7889989878654
\img\RogerAckroyd.jpg	Who Killed Roger Ackroyd?	8877766655543
\img\StrategicMoves.jpg	Strategic Moves	5566677788676

The next step is to activate a command button by editing the columns of gvDeleteBooks. If you need help, review the work we did at the bottom of Page 10.

Hint: Add three bound fields and a command button.



Just make sure the DataKeyName is set to **ISBN**.

Double click the gridview to raise the selectedIndexChanged event and add the bolded code below.

```
protected void gvDeleteBooks_SelectedIndexChanged(…
    {
        Response.Write(gvDeleteBooks.SelectedValue.ToString());
    }
```

Once the correct ISBN appears on the top of the page, we can add the SQL statements.

```
protected void gvDeleteBooks_SelectedIndexChanged(object sender, EventArgs e)
    {
       //Response.Write(gvDeleteBooks.SelectedValue.ToString();
        string sqlCommand = "DELETE FROM " + ownerID + "BOOKS WHERE ISBN = " +
gvDeleteBooks.SelectedValue.ToString();

        myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref
lblErrorMessage);

      //redisplay books
        sqlCommand = "SELECT IMAGE, TITLE, ISBN FROM " + ownerID + "BOOKS";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref
lblErrorMessage);

    }
```

The one thing we must consider is if a BestSeller happens to be on our discontinued list.

```
protected void gvDeleteBooks_SelectedIndexChanged(…
{
       //Response.Write(gvDeleteBooks.SelectedValue.ToString());
       string sqlCommand = "DELETE FROM " + ownerID + "BOOKS WHERE ISBN = " +
gvDeleteBooks.SelectedValue.ToString();
       myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref lblErrorMessage);

       // we need to remove from Best Seller if the book is there.
       try
       {
             sqlCommand = "DELETE FROM " + ownerID + "BESTSELLER WHERE ISBN = " +
gvDeleteBooks.SelectedValue.ToString();
```

```
            myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref
lblErrorMessage);
        }
        catch { }

        //redisplay books
        sqlCommand = "SELECT IMAGE, TITLE, ISBN FROM " + ownerID + "BOOKS";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDeleteBooks, ref lblErrorMessage);
```

Ed Note: If your page does not refresh after a delete, make sure the try catch block is before the redisplay code.

Can we show the images or do we not include them?

The correct technique is to insert a template as we used in Default.aspx. First we need to copy the following method from Default.aspx or Admin.aspx.

```
protected string FormatImageUrl(string url)
    {
        if (url != null && url.Length > 0)
            return ("~/" + url);
        else return null;
    }
```

Then we modify the source code in removeBooksFromInventory.aspx and look for

```
<asp:GridView ID="gvDeleteBooks" runat="server" AutoGenerateColumns="False" DataKeyNames="ISBN"
OnSelectedIndexChanged="gvDeleteBooks_SelectedIndexChanged">
```

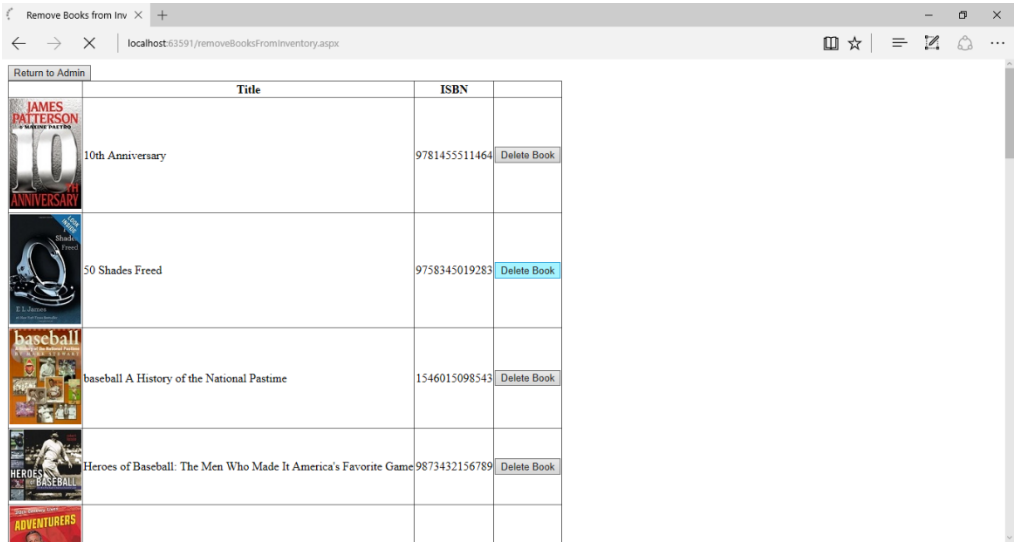
Ed Note: You will see a template field once you have modified the gridview object.

by replacing the first bound field with the following template code:

```
 <asp:TemplateField>
     <ItemTemplate>
       <asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string) FormatImageUrl( (string)
Eval("Image")) %>' />
     </ItemTemplate>
  </asp:TemplateField>
```

Ed Note: This code appears a lot in the course of our project. There are several sets of code which seem to keep re-appearing. Perhaps we can place some of this code in a code book and save it for future use.

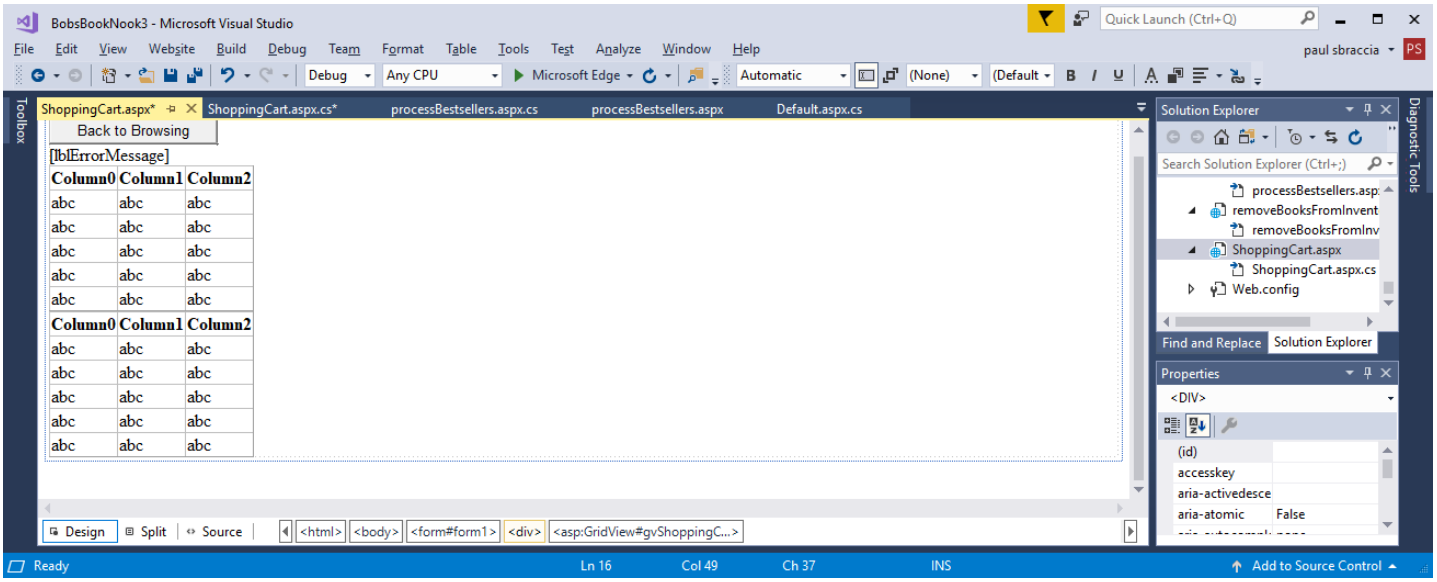
Just about to delete 50 Shades of Freed.



Making the ShoppingCart More Responsive.

- Add a Checkout button which for now, clears the items in sessions variables and the gridview and returns the browser back to Default.aspx.
- On the ShoppingCart page
  - display each book that has been selected.
  - The gridview will show the image, title, author and ISBN.
  - You will add a button to "Place back on the shelf" which removes the record from the shopping cart gridview by removing it from the sessions variable.

To Start: Add btnCheckout, lblErrorMessage just below btnBackToBrowaer and **gvShoppingCart** and **gvDisplay** below that. Be sure to set the label visible option to **false** and set the **dataKeyNames** to **ISBN** for **gvShoppingCart**. and **gvDisplay**.



At the top of the shoppingCart.aspx.cs, add the following bold code to implement ownerID.

```
public partial class ShoppingCart : System.Web.UI.Page
{
    string ownerID;

    protected void Page_Load(object sender, EventArgs e)
    {
        ownerID = getOwnerID.getID(Server.MapPath("~/App_Data/"));
        try
        {
```

We now have access to ownerID throughout the modifications necessary to the shoppingCart.

Filling the GridView:

Do you remember my comment back in Part 2? Where the duplicate session variables made no difference? Hind Site is always 20/20. In testing the solution as originally planned, I ended up displaying a previously deleted item. Upon further Internet research, it was stated that "Session Variables are unpredictable at best, and anyone who builds a web site based on [them] is flirting with disaster. Use a database table to hold Shopping Cart data."

Upon further discussion with a potential customer, [my sister], the only way we could figure on maintaining a database without having users login just because they put something in their shopping cart leads to the following solution which is placed in Page_Load.

Before we can fill the gridView:

We have to face the fact that often times pages seem to be reloaded before exiting, or some other page action that messes with our logical solutions. In this case, we need to look at what we have and how can we prevent duplicate pageload activities.

What we know is that we have a QueryString value if the page is called from Default, and no value if someone just tries to load the shoppingcart page out of the blue, or we set the shoppingcart as the start page.

Make the following change to Page_Load in ShoppingCart.aspx.cs.

```
//try
//{
////Save session variable
//Session[Session.Count.ToString()] = Request.QueryString["id"];
////display session variable
//for (int i = 0; i < Session.Count; i++)
//{
//    Response.Write("<br />" + Session[i.ToString()].ToString());
//}
```

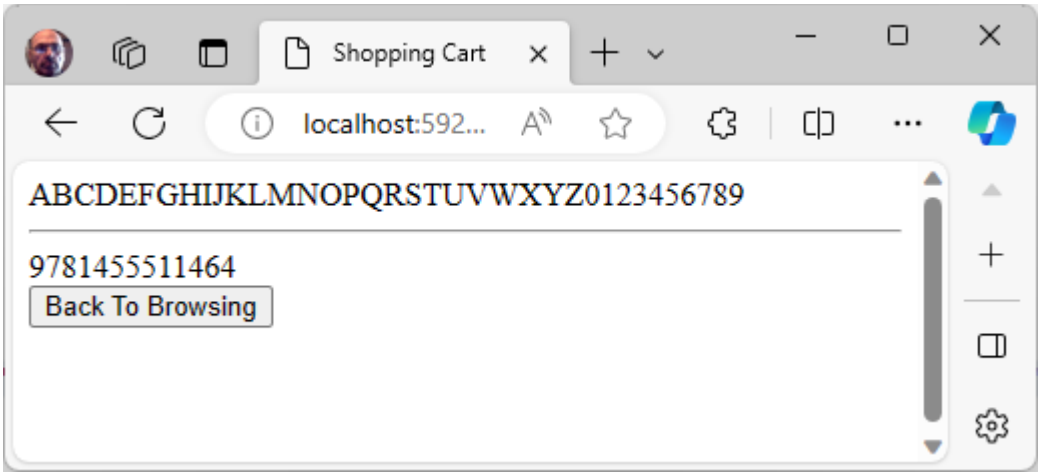
Is replaced with an if else structure which is more dependable than working with session variables.

```
string strQS = Request.QueryString["id"];

if (strQS == null)
{
     Response.Redirect("Default.aspx");
}
   else
 {
     CreateSessionVariable(strQS);
 }


private void CreateSessionVariable(string strQS)
 {
     Response.Write(strQS+ "\r\n");
 }
```

Since we do not yet have the user information such as first name, last name, creditcard number, email, etc, we need to use the random function to generate a 20 character length newTempUserID. This new id will be used to create a temporary table within our database to hold each of the ISBNs.

Make the following change (bold code only) to `CreateSessionVariable` in ShoppingCart.aspx.cs.

```
private void CreateSessionVariable(string strQS)
{

    string newTempUserID = "";
    string tempUserSource = "A" + "B" + "C" + "D" + "E" + "F" + "G" + "H" + "I" +
"J" + "K" + "L" + "M" + "N" + "O" + "P" + "Q" + "R" + "S" + "T" + "U" + "V" + "W" +
"X" + "Y" + "Z" + "0" + "1" + "2" + "3" + "4" + "5" + "6" + "7" + "8" + "9";
    //remove after testing
    Response.Write(tempUserSource + "<hr>");


    Response.Write(strQS+ "\r\n");
}
```



Note that the ISBN number will change depending on book selected.

Creating a unique shopperID

Add the following BOLD code just below Response.Write(tempUserSource + "<hr>");

```
string newTempUserID = "";
string tempUserSource = "A" + "B" + "C" + "D" + "E" + "F" + "G" + "H" + "I" + "J" + "K" + "L" + "M" +
"N" + "O" + "P" + "Q" + "R" + "S" + "T" + "U" + "V" + "W" + "X" + "Y" + "Z" + "0" + "1" + "2" + "3" +
"4" + "5" + "6" + "7" + "8" + "9";
Response.Write(tempUserSource + "<hr>");

if (System.Web.HttpContext.Current.Session["tempUserID"] != null)
   {
   newTempUserID = Session["tempUserID"].ToString();
   }
else
   {
       Random r = new Random();
       int index = r.Next(0, 26);//Makes sure first character is a letter
       newTempUserID += tempUserSource.Substring(index, 1);
       for (int i = 2; i < 21; i++)
       {
           index = r.Next(0, 36);
```

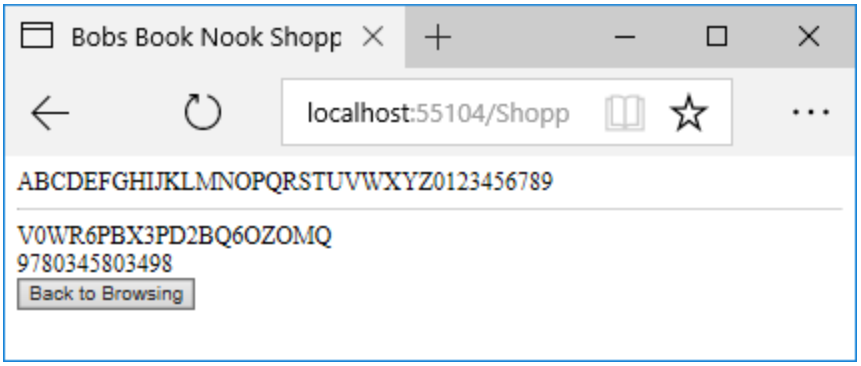
```
            //Must be += or we only capture the last letter.
            newTempUserID += tempUserSource.Substring(index, 1);
            Response.Write(newTempUserID);
        }
  Response.Write(newTempUserID);
   }
Response.Write(newTempUserID + "<HR>");

//Show ISBN of new Selection
Response.Write(strQS + "\r\n");
```

As you can see in the figure to the right, we are adding a random character from tempUserSource. Using base36 numbering we can be sure that no two newTempUserID will be the same. What we are doing with the newTempUserID? Using it to make a temporary table containing individual user shopping cart selections.

Modify the code in CreateSessionVariable as shown below to save the newTempUserID to a sessions variable. *Ed Note: Much the same way we used cookies in Homework 4.*

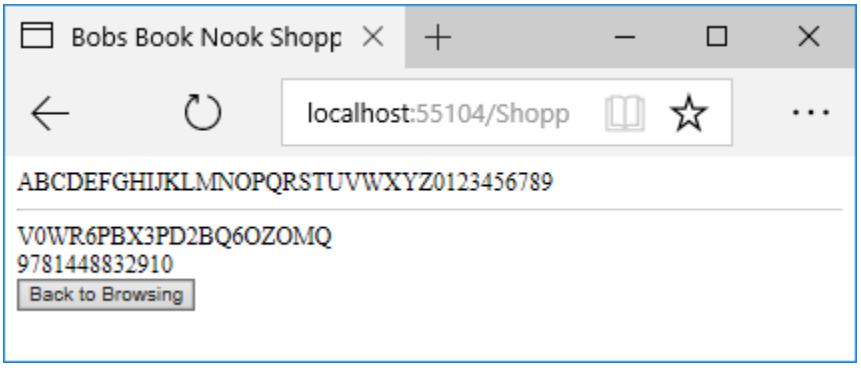
```
string newTempUserID = "";
string tempUserSource = "A" + "B" + "C" + "D" + "E" + "F" + "G" + "H" + "I" + "J" + "K" + "L" +
"M" + "N" + "O" + "P" + "Q" + "R" + "S" + "T" + "U" + "V" + "W" + "X" + "Y" + "Z" + "0" + "1" +
"2" + "3" + "4" + "5" + "6" + "7" + "8" + "9";
 //remove after testing
 // Response.Write(tempUserSource + "<hr>");

 if (Session["tempUserID"] == null)
 {
     Random r = new Random();
     int index = r.Next(0, 26);//Makes sure first character is a letter
     newTempUserID += tempUserSource.Substring(index, 1);
     for (int i = 2; i < 21; i++)
     {
         index = r.Next(0, 36);
         //Must be += or we only capture the last letter.
         newTempUserID += tempUserSource.Substring(index, 1);
         // Response.Write(newTempUserID + "<br/>");
     }

   Session["tempUserID"] = newTempUserID;
 }
 Response.Write(Session["tempUserID"].ToString() + "<br/>");
 Response.Write(strQS + "\r\n");
```

First Selection:



Second Selection:

Notice that the VOWR6… remains the same where the ISBN number changes with each trip back to the selection page.



Near the bottom of CreateSessionVariable method, we need to create the table just after we save the session variable:

Add the call to createGridView as shown below and use refraction to create the method.

```
          Session["tempUserID"] = newTempUserID;
          createTempTable();
        }
        Response.Write(Session["tempUserID"].ToString() + "<br/>");
        Response.Write(strQS + "\r\n");
```

```
private void createTempTable()
{
    string sqlCommand = "CREATE TABLE " + Session["tempUserID"] + " (ISBN
CHAR(14))";
    myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);
}
```

What do we do next? We add a way to clear the Session variable so that we can change tables for the next user (and to make things easier for the database administrator (your instructor) by having you delete old tables so s/he does not have spend two hours at the end of the semester cleaning things up!))

Add a button titled btnCheckout. Place any where you think that it is functional and looks appropriate.

Double click the new button to create the module and add the two likes of code which remove the Session variable and redirects to Default.aspx.

```
protected void btnCheckout_Click(object sender, EventArgs e)
{
      string sqlCommand = "DROP TABLE " + Session["tempUserID"] ;

      myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);

      Session["tempUserID"] = null;

      Response.Redirect("Default.aspx");
}
```

Next Step? Add the ISBN to the table and display the current table. At the bottom of Page_Load add a call to addToAndShowGridView as follows:

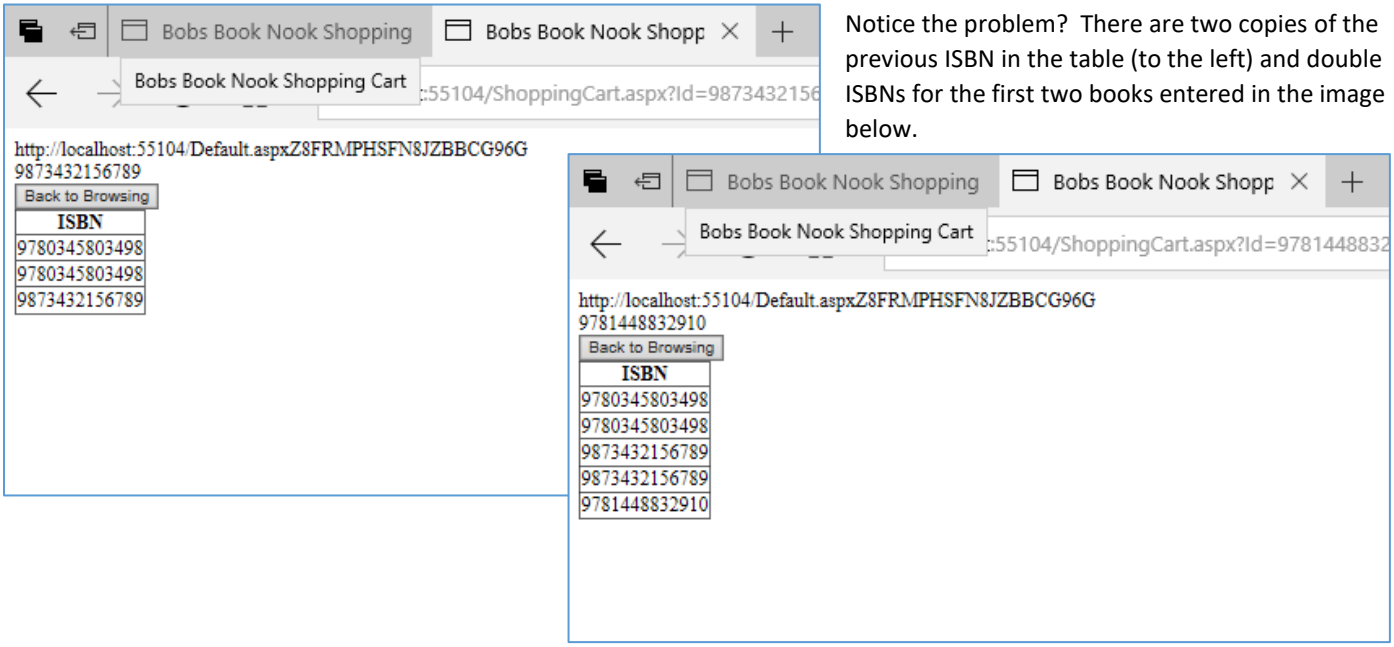
```
else
{
      CreateSessionVariable(strQS);
      addToAndShowGridView(strQS);
}
```

Create the method by using refactoring and add the following code:

```
private void addToAndShowGridView(string strQS)
{
    string sqlCommand = "INSERT INTO " + Session["tempUserID"] + " VALUES(" + strQS
+ ")";

    myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);
```

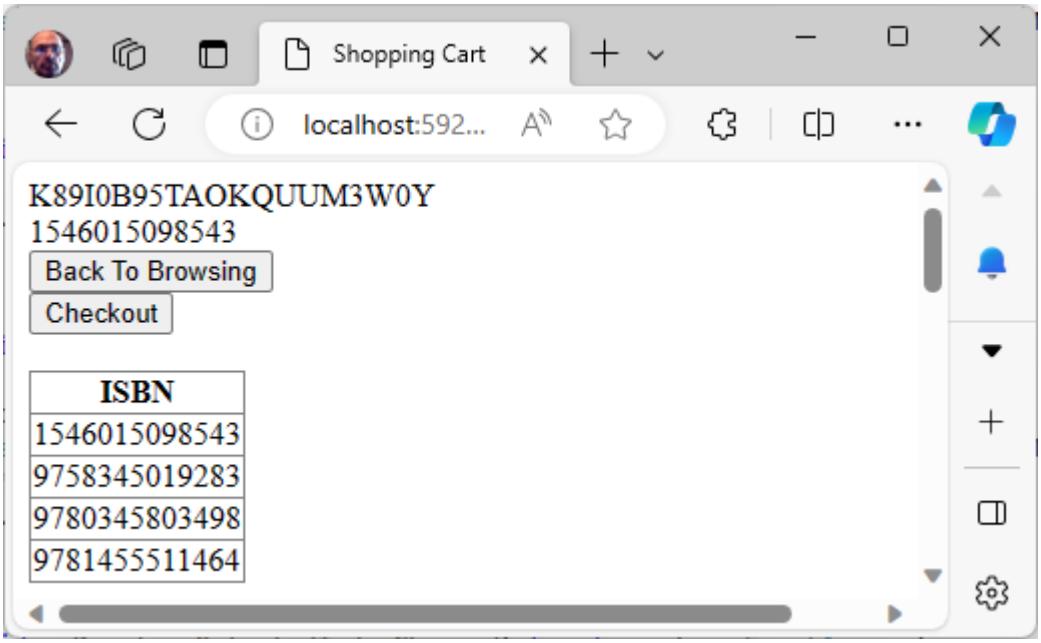
```
    sqlCommand = "SELECT DISTINCT ISBN FROM " + Session["tempUserID"];

    myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);

}
```

Reading the code you can see that we insert the data into a table using the session variable and the QuerryString value, then we complete a SELECT statement to view the results.



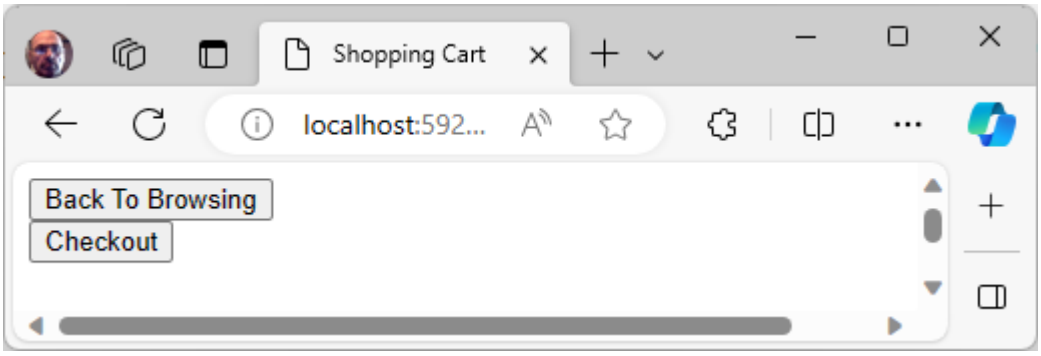
Notice the problem? There are two copies of the previous ISBN in the table (to the left) and double ISBNs for the first two books entered in the image below.

Before we continue, this is a GREAT time to remove the extra clutter atop our work area. In the figure below, comment out the first two lines written from Response.Write statements, and make sure the top gridview.visible is set to false.



You may need to add the following line of code:

```
private void addToAndShowGridView(string strQS)
 {
     string sqlCommand = "INSERT INTO " + Session["tempUserID"] + " VALUES(" + strQS + ")";
     myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref lblErrorMessage);
     sqlCommand = "SELECT DISTINCT ISBN FROM " + Session["tempUserID"];
     myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref lblErrorMessage);
      gvShoppingCart.Visible = false;

 }
```



Show me my Selections

In **ShoppingCart.aspx** source, we need to modify the code behind **gvDisplay** making the following changes in ShoppingCart.aspx source view. The reason is we only monitor temporarily the ISBNs in gvShoppingCart, and would create havoc attempting to include Image, Title and other fields where an error would confuse us and the GUI. Let's use a separate gridview for clarity of purpose.

Replace the current HTML code for ID="gvDisplay" with the following.

```
<asp:GridView ID="gvDisplay" runat="server" AutoGenerateColumns="False">
  <Columns>
    <asp:BoundField DataField="Image" HeaderText="Image" />
    <asp:BoundField DataField="Title" HeaderText="Title" />
    <asp:BoundField DataField="Author" HeaderText="Author" />
    <asp:BoundField DataField="ISBN" HeaderText="ISBN" />
    <asp:CommandField ButtonType="Button" ShowCancelButton="False" ShowSelectButton="True"
SelectText="Place Back on Shelf" />
  </Columns>
</asp:GridView>
```

At the very bottom of Page_Load insert a call to viewGvDisplay and create the method.

```
 {
 CreateSessionVariable(strQS);
addToAndShowGridView(strQS);
viewGvDisplay();
 }
```

```
private void viewGvDisplay()
{
      //throw new NotImplementedException();
}
```

We have to use the contents of the temporary table with an inner join to fill **gvDisplay**.

```
private void viewGvDisplay()
{
string sqlCommand;

//throw new NotImplementedException();
try
{
  string table1 = ownerID + "BOOKS";
  string table2 = Session["tempUserID"].ToString();

 sqlCommand = "SELECT IMAGE, AUTHOR, TITLE, " + table1 + ".ISBN from " + table1 + "
INNER JOIN " + table2 + " ON " + table1 + ".ISBN = " + table2 + ".ISBN ";

  myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblErrorMessage);
}

catch
{
      Response.Redirect("Default.aspx");
}

}
```

Within the definition for gvDisplay replace the
`<asp:BoundField DataField="Image" HeaderText="Image" />` statement with the following:

```
<asp:TemplateField>
<ItemTemplate>
<asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string) FormatImageUrl( (string)
Eval("Image")) %>' />

</ItemTemplate>
</asp:TemplateField>
```

And we need to copy the following as a separate method placed in the aspx.cs file.

```
protected string FormatImageUrl(string url)
{
      if (url != null && url.Length > 0)
            return ("~/" + url);
      else return null;
}
```

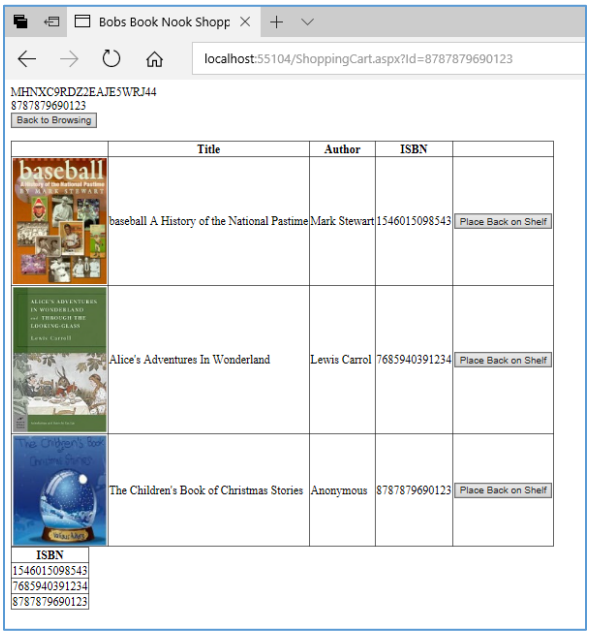
From Default.aspx or Admin.aspx or one of the other aspx.cs pages where we have placed this code.

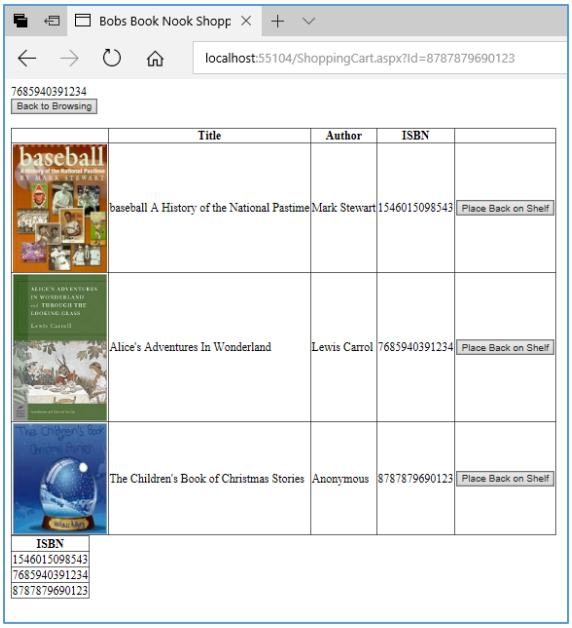
Double click gvDisplay to expose the selectedIndexChanged method.

Add the bold code.

```
protected void gvDisplay_SelectedIndexChanged(…
{
        Response.Write(gvDisplay.SelectedValue.ToString());
}
```

At this point when we click on Place Back on Shelf, we will see the ISBN of the item we clicked on.





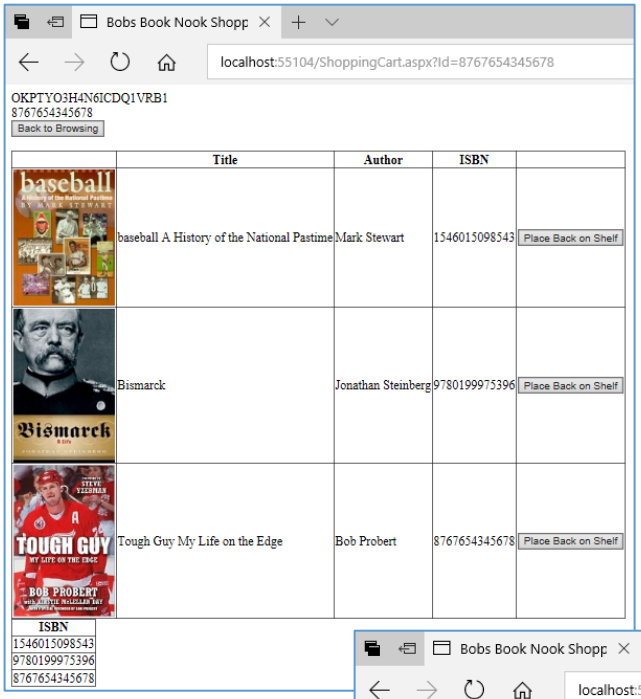
Before we select a Book to remove from our basket. After we click on Alice.

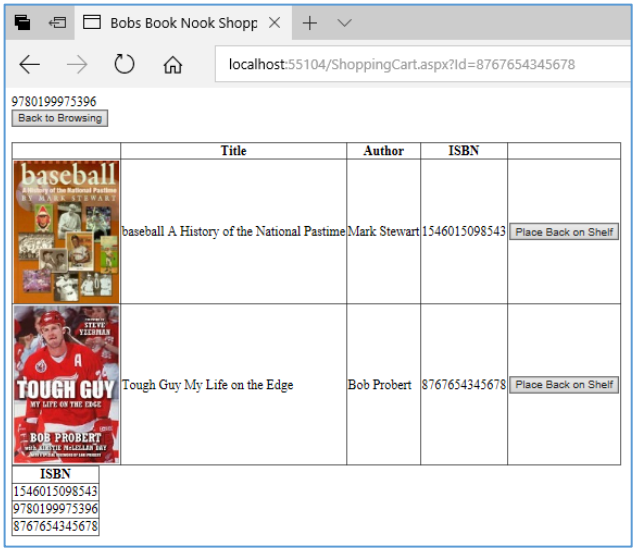
Notice that the ISBN of Alice is showing up on the top of the page.

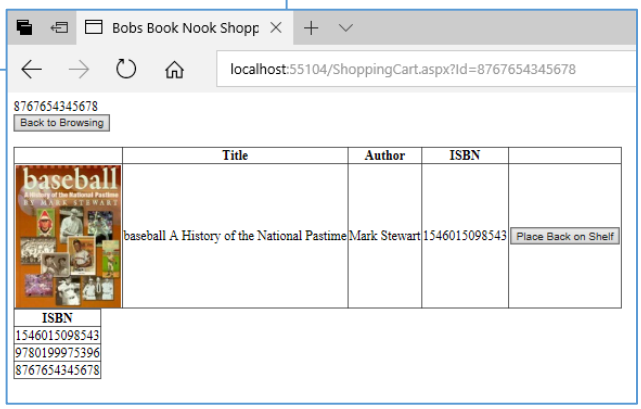
Removing Items

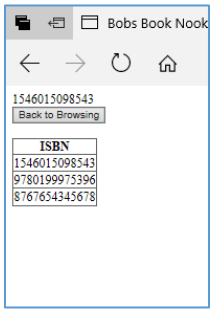
We simply need to remove the item from the temporary table and re-run the inner join statement.

```
protected void gvDisplay_SelectedIndexChanged(object sender, EventArgs e)
{
        //  Response.Write(gvDisplay.SelectedValue.ToString());
        string sqlCommand;
        string table1 = ownerID + "BOOKS";
        string table2 = Session["tempUserID"].ToString();

        sqlCommand = "DELETE FROM " + table2 + " WHERE ISBN = " + gvDisplay.SelectedValue;
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblErrorMessage);

       // and use a call to viewGvDisplay to repopulate gvDisplay.

        viewGvDisplay();
}
```









Before you check your program for submission, please make sure the following Response.Write statements are commented and that the visible property of gvShoppingCart is set to invisible, even if you have to set it by hand after a call to myDatabaseConnection.

```
//Response.Write(Session["tempUserID"].ToString() + "<br/>");

//Response.Write(Session["tempUserID"].ToString() + "<br/>");
//Show ISBN of new Selection
//Response.Write(Request.QueryString["id"] + "<br/>");
```

CheckOut

Since we are (in the next project) creating an "invoice" page, all we need to do for now is to code btnCheckout to drop table2, clear the session data, and return to Default.aspx. You can place btnCheckOut where ever you think is best.

```
protected void btnCheckout_Click(…
{
try {
      string sqlCommand;
      string table2 = Session["tempUserID"].ToString();

      sqlCommand = "DROP TABLE " + table2;
      myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);

      Session["tempUserID"] = null;
      }

catch{}

Response.Redirect("Default.aspx");
}
```

To make sure that gvShoppingCart does not show, we need to search for any place where we ref gvShoppingCart. We can comment out the select statement and the call to myDatabaseConnection or just reset gvShoppingCart visible to false just after the myDatabaseConnection statement.

```
private void addToShoppingList(string userID, string ISBN)
{
       //throw new NotImplementedException();
       string sqlCommand = "INSERT INTO " + userID + " VALUES(" + ISBN + ")";
       myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref lblErrorMessage);
       //sqlCommand = "SELECT * FROM " + userID;
       //myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref lblErrorMessage);
}
```

As you most likely come to realize, we are reusing both C# code and HTML code. Much of what we will be doing in many of our projects (here and in the future) will boil down to what will become very familiar code blocks, procedures and class files.

If you had a code based instructor for ITWP1000 you would have been told to start a code book (electronic is fine). Keep copies of code which begins to re-appear more than once in your work. You do not have to turn one in, though it is a good idea to consider this for future revisions to this content.

- Do make sure your application works.
- Remove any and all extra response.write statements which we used for debugging.
- Upload to the server.
- Be sure to upload the contents of App_Code as they have changed with this assignment.

- Execute the application by entering the URL to Default.aspx and then by entering the URL to Admin.aspx.
- You are free to modify the layout of any page in this project just as long as the resulting submission works as expected.
- Post both URLs (Default.aspx and Admin.aspx). Place Admin.aspx in the submissions box and add Default.aspx to the notes section.

That's it for Project 3.